InsaCloud

Fiche technique : la sécurité

Ce que la plateforme protège, contre quoi,
par quel mécanisme — et ce qu'elle ne protège pas

Une plateforme qui loue des machines Linux avec un accès `root` et des ports ouverts sur le réseau est, par construction, une surface d'attaque. Ce document prend chaque mécanisme de défense un par un et répond toujours aux mêmes quatre questions : **quelle menace** il traite, **quelle parade** il oppose, **où il se trouve** dans le code avec ses valeurs exactes, et **ce qu'il ne couvre pas**.

Cette dernière question n'est pas une concession : une défense dont on ignore la limite donne une fausse assurance, et une fausse assurance est pire qu'une absence de défense.

Mohamed Aziz Baoueb • Youssef Farjallah • Elyes Lemhanni

Projet 3 — *Outils de déploiement de plateformes*
INSA Centre-Val de Loire, STI 4A FISE, 2026/2027 — Dr. Nadia Fettah

2 novembre 2026

Table des matières

1 Le modèle de menace 1

2 Couche 2 — L'identité 4

3 Couche 3 — La session et le navigateur 8

4 Couche 4 — Les secrets 12

5 Couche 5 — L'accès aux machines louées 17

6 Couche 6 — Le durcissement des conteneurs 22

7 Couche 7 — Le contrôleur et les nœuds 28

8 Couche 8 — Les données et le réseau 33

9 Couche 9 — La chaîne DevSecOps 37

10 Couche 10 — Les limites assumées 42

CHAPITRE 1

Le modèle de menace

Avant de lister des défenses, il faut dire ce qu'on défend et contre qui. Sans cela on empile des mécanismes à la mode sans savoir lesquels comptent.

Ce que la plateforme expose

InsaCloud n'est pas un site vitrine. Elle expose délibérément trois choses que l'on ne met normalement pas sur un réseau :

— **Des comptes `root`**, un par machine louée, avec leur mot de passe, remis à des utilisateurs qu'on ne connaît pas.

— **Une plage de ports ouverts** — 8000 à 9000 — derrière lesquels répondent des serveurs SSH, des terminaux web et des bureaux graphiques.

— **Un démon Docker** sur chaque nœud, c'est-à-dire un service dont le contrôle équivaut à être `root` sur la machine hôte.

> **Notion — Pourquoi le démon Docker est l'actif le plus sensible**
>
> Quiconque peut parler au démon Docker peut lancer un conteneur privilégié qui monte le disque de l'hôte. Il n'y a pas d'escalade à trouver : c'est le fonctionnement normal de Docker. « Accès au démon Docker » et « accès administrateur de la machine » sont donc deux formulations de la même chose, et toute décision d'architecture doit partir de là.

Les quatre adversaires

Qui	Ce qu'il cherche, et ce qui l'arrête
Le robot d'Internet	Il balaie les ports, tente des mots de passe SSH et des dizaines de milliers de chemins HTTP connus. C'est l'adversaire le plus *fréquent* et le plus bête. Parades : authentification par clé, limitation des tentatives, en-têtes et politique de contenu stricts.
L'utilisateur curieux	Un étudiant légitime qui essaie de voir les machines d'un autre, de prolonger sa location sans payer le temps, ou de sortir de son conteneur. C'est l'adversaire le plus *probable* dans un contexte pédagogique. Parades : cloisonnement par propriétaire vérifié en base, capacités minimales dans le conteneur, quotas.
Le locataire malveillant	Il loue une machine en règle, puis s'en sert comme tremplin : scanner le réseau interne, atteindre le contrôleur, miner, attaquer l'extérieur. Il est *déjà* `root` dans son conteneur — c'est le service rendu. Parades : durcissement du conteneur, réseau dédié, limites de ressources, durée de vie courte.
Celui qui lit le dépôt	Le code est sur GitHub. Un secret commis par erreur, même retiré ensuite, reste dans l'historique. Parades : `gitleaks` sur tout l'historique à chaque envoi, Ansible Vault, fichiers de clés exclus du dépôt.

Ce qui est hors périmètre, et pourquoi

Un modèle de menace honnête nomme aussi ce qu'il abandonne.

Un attaquant qui a déjà `root` sur le contrôleur a gagné : il lit la base, les clés, et pilote tous les nœuds. Aucune des mesures décrites ici ne le gêne. La défense contre ce cas est organisationnelle (qui a accès à la machine physique), pas logicielle.

Une évasion de conteneur par une faille du noyau. Les conteneurs partagent le noyau de l'hôte ; une vulnérabilité du noyau les traverse. Le durcissement réduit la surface (capacités retirées, `no-new-privileges`) mais ne crée pas la frontière qu'offrirait une machine virtuelle. C'est le compromis assumé de la conteneurisation, et l'énoncé le tranche en imposant Docker.

Le déni de service distribué. Les limites de ressources par conteneur protègent les *voisins* d'un locataire gourmand ; elles ne protègent pas la plateforme d'une attaque volumétrique venue de l'extérieur, qui relève de l'hébergeur.

À retenir

Trois actifs à protéger : le **démon Docker** (équivalent à `root` sur l'hôte), les **secrets** (clés, mots de passe des machines), et le **cloisonnement entre locataires**. Tout le reste en découle.

Les dix couches de défense

Le reste du document suit l'ordre d'une requête, du navigateur jusqu'au conteneur :

#	Couche	Ce qu'elle empêche
2	Identité et authentification	qu'on se fasse passer pour un utilisateur
3	Session et navigateur	qu'on détourne une session ouverte
4	Secrets	qu'un secret traîne en clair quelque part
5	Accès aux machines louées	qu'on entre dans la machine d'un autre
6	Durcissement des conteneurs	qu'un locataire sorte de sa machine
7	Contrôleur et nœuds	qu'on prenne la main sur un démon Docker
8	Données et réseau	qu'on corrompe l'état ou qu'on écoute le trafic
9	Chaîne d'intégration	qu'une faille entre dans le dépôt sans qu'on le voie
10	Limites assumées	qu'on se croie protégé là où on ne l'est pas

CHAPITRE 2

Couche 2 — L'identité

Quatre mécanismes pour qu'un compte ne puisse pas être pris par quelqu'un d'autre.

Le hachage des mots de passe

mesure 1 · app.py

La menace

Une base de données finit par fuiter : sauvegarde oubliée, injection SQL, disque revendu. Si elle contient les mots de passe en clair, tous les comptes tombent — et, comme les gens réutilisent leurs mots de passe, des comptes ailleurs aussi.

La parade

On ne stocke jamais le mot de passe, mais son **empreinte**, calculée par une fonction à sens unique et *lente*. La lenteur est la propriété recherchée : elle ne gêne pas l'utilisateur qui se connecte une fois, mais elle ruine l'attaquant qui doit en tester des milliards.

Dans le code

`generate_password_hash` de Werkzeug, qui applique **PBKDF2-SHA256** avec un sel aléatoire par compte et un grand nombre d'itérations. Le sel rend les tables précalculées inutilisables : deux utilisateurs ayant le même mot de passe ont deux empreintes différentes.

```
user_id = db.create_user(username, generate_password_hash(password),
    ↪ email or None)
```

Piège — Comparer en temps constant, et même quand le compte n'existe pas

Une vérification naïve répond *plus vite* quand l'identifiant est inconnu — il n'y a pas d'empreinte à calculer. En chronométrant les réponses, on énumère donc les comptes existants sans jamais réussir une connexion. La parade tient en une ligne : quand l'utilisateur n'existe pas, on calcule quand même une empreinte factice, pour que les deux cas coûtent le même temps.

```
ok = user is not None and check_password_hash(user["password_hash"
    ↪ ], password)
```

```
# meme cout quand le compte n'existe pas : on ne doit pas pouvoir les enumerer
check_password_hash(generate_password_hash("x"), password) if user is None else None
```

Ce que ça ne couvre pas

Un mot de passe faible reste faible, même bien haché. D'où la mesure suivante.

La politique de mot de passe

mesure 2 · `config.py`

La menace

Les attaques par dictionnaire ne cassent pas la cryptographie : elles essaient `123456`, `password`, `azertyuiop`. Un seul compte faible suffit à entrer.

La parade

Une longueur minimale et le refus d'une liste de mots de passe courants, à l'inscription.

Dans le code

```
PASSWORD_MIN_LENGTH = int(os.environ.get("INSACLOUD_PASSWORD_MIN", "10"))
COMMON_PASSWORDS = {"password", "motdepasse", "123456789", "1234567890",
                    "azertyuiop", ...}
```

Dix caractères, pas huit : la règle classique des huit caractères date d'une époque où le matériel était mille fois plus lent.

Ce que ça ne couvre pas

Aucune politique n'empêche `Soutenance2026!` — conforme, et devinable par qui connaît le contexte. La vraie défense contre le devinage ciblé est la suivante.

La limitation des tentatives

mesure 3 · `app.py`

La menace

Avec un formulaire de connexion public, un robot teste des milliers de combinaisons par minute. Sans limite, il finit par trouver.

La parade

Un compteur de tentatives échouées, sur **deux axes** et dans une fenêtre glissante.

Dans le code

```
LOGIN_WINDOW_MINUTES    = 15   # la fenetre glissante
LOGIN_MAX_FAILURES_USER = 5    # par compte vise
LOGIN_MAX_FAILURES_IP   = 20   # par adresse source
```

Les deux axes traitent deux attaques différentes :

— **par compte** — l'attaquant s'acharne sur `admin` depuis mille adresses : le compteur du compte le bloque ;

— **par adresse** — l'attaquant essaie un mot de passe très probable sur mille comptes différents (*password spraying*), ce qui ne déclenche aucun compteur par compte : le compteur par adresse le bloque.

Chaque tentative est enregistrée en base avec son horodatage et son adresse source ; les entrées de plus de vingt-quatre heures sont purgées à chaque connexion réussie, pour que la table ne grossisse pas indéfiniment.

> **À retenir**
>
> Deux compteurs valent mieux qu'un : un seul compteur par compte ne voit pas le *password spraying*, et un seul compteur par adresse ne voit pas l'attaque distribuée. C'est la combinaison qui couvre les deux.

Ce que ça ne couvre pas

Un attaquant disposant de beaucoup d'adresses et visant beaucoup de comptes passe sous les deux seuils. À l'échelle de ce projet c'est acceptable ; en production on ajouterait un CAPTCHA après quelques échecs, ou une temporisation croissante.

La validation des entrées

mesure 4 · `config.py`

La menace

Toute donnée venue de l'utilisateur finit quelque part : dans une requête SQL, dans un nom de conteneur, dans une page HTML, dans une ligne de commande. Si elle n'est pas contrainte, elle y fait ce qu'elle veut.

La parade

Des expressions régulières **ancrées** — `^` et `$` — qui décrivent ce qui est *autorisé*, jamais ce qui est interdit. Une liste d'interdits est toujours incomplète ; une liste d'autorisés ne l'est jamais.

Dans le code

```
USERNAME_RE   = re.compile(r"^[a-zA-Z0-9][a-zA-Z0-9_.-]{2,31}$")  #
    ↪ compatible Docker
EMAIL_RE      = re.compile(r"^[^@\s]{1,64}@[^@\s.]+(\.[^@\s.]+)+$")
```

```
SSH_PUBKEY_RE = re.compile(r"^(ssh-ed25519|ssh-rsa|ecdsa-sha2-nistp
    ↪ (256|384|521)"
                           r"|sk-ssh-ed25519@openssh\.com) [A-Za-z0
                               ↪ -9+/]+=*"
                           r"( [^\r\n]{0,128})?$")
MIN_DURATION_MINUTES = 1
MAX_DURATION_MINUTES = 120
MAX_INSTANCES_PER_USER = 3
```

Le format de l'identifiant n'est pas arbitraire : il est choisi pour être **compatible avec les noms de conteneurs Docker**, puisqu'il s'y retrouve. Un identifiant contenant une espace ou un point-virgule serait refusé bien avant d'atteindre la ligne de commande.

> **Notion — La requête est bornée avant même d'être lue**
>
> `MAX_CONTENT_LENGTH = 16 * 1024` : Flask rejette toute requête de plus de seize kilooctets. Aucun formulaire légitime de la plateforme n'en approche — le plus gros est une clé publique SSH, quelques centaines d'octets. Cette borne coupe court aux tentatives d'épuisement mémoire par un corps de requête géant, avant que le code applicatif ne voie quoi que ce soit.

Ce que ça ne couvre pas

La validation protège le format, pas le sens. Une durée de 120 minutes est valide ; demander trois machines de 120 minutes toutes les deux minutes reste possible, et c'est le quota (`MAX_INSTANCES_PER_USER`) qui l'arrête.

CHAPITRE 3

Couche 3 — La session et le navigateur

L'utilisateur est authentifié. Reste à empêcher qu'on lui vole sa session, ou qu'on la lui fasse utiliser à son insu.

Le cookie de session

mesure 5 · `config.py`

La menace

Après la connexion, c'est le cookie qui prouve l'identité. Le voler, c'est devenir l'utilisateur sans jamais connaître son mot de passe.

La parade

Quatre attributs, chacun fermant une porte différente.

Dans le code

```
SESSION_COOKIE_HTTPONLY    = True              # inaccessible a
    ↪ JavaScript
SESSION_COOKIE_SAMESITE    = "Strict"          # jamais envoye depuis un
    ↪ autre site
SESSION_COOKIE_SECURE      = HTTPS             # transmis seulement en
    ↪ TLS
SESSION_COOKIE_NAME        = "insacloud_session"
PERMANENT_SESSION_LIFETIME = timedelta(hours=2)
```

Attribut	La porte qu'il ferme
`HttpOnly`	Un script injecté dans la page ne peut pas lire le cookie : une faille XSS ne se transforme pas en vol de session.
`SameSite=Strict`	Le navigateur n'envoie pas le cookie quand la requête vient d'un autre site. C'est la défense structurelle contre le CSRF, celle qui agit avant même que la requête parte.
`Secure`	Le cookie ne circule jamais en clair. Activé dès que le site est en HTTPS — désactivé en développement local, sinon le cookie ne serait jamais émis et rien ne fonctionnerait.
`2 heures`	Une session oubliée sur un poste de salle TP finit par expirer.

Piège — Une session Flask est signée, pas chiffrée

C'est la méprise la plus répandue. Le contenu du cookie est encodé en base64 : n'importe qui peut le *lire*. Ce que la clé secrète garantit, c'est qu'on ne peut pas le *modifier* sans invalider la signature.
Conséquence directe sur le code : la session ne contient que l'**identifiant** de l'utilisateur. Jamais son rôle, jamais un jeton, jamais un mot de passe. Tout le reste est relu en base à chaque requête.

Ce que ça ne couvre pas

Un cookie volé reste valable jusqu'à son expiration : il n'est lié ni à l'adresse IP ni au navigateur. Lier une session à son adresse casserait les connexions mobiles et n'arrêterait qu'un attaquant peu motivé.

Les jetons CSRF

mesure 6 · tous les formulaires

La menace

Un site tiers contient une image ou un formulaire qui pointe vers `/instances/42/supprimer`. L'utilisateur, connecté à InsaCloud dans un autre onglet, le visite : son navigateur joint son cookie, et la requête part avec son identité.

La parade

Chaque formulaire porte un jeton aléatoire lié à la session ; le serveur refuse toute requête qui modifie l'état sans ce jeton. Un site tiers ne peut pas le deviner — il ne peut pas lire la page qui le contient, la politique de même origine l'en empêche.

Dans le code

`WTF_CSRF_ENABLED = True`, y compris en configuration de test : la protection CSRF reste donc exercée par la suite de tests, et une régression qui la désactiverait ferait échouer la chaîne d'intégration.

À retenir

`SameSite=Strict` et le jeton CSRF font le même travail à deux endroits différents — le navigateur et le serveur. On garde les deux : le premier dépend du navigateur de l'utilisateur, le second de personne.

Ce que ça ne couvre pas

Le CSRF ne protège pas d'un script qui s'exécute *dans* la page : celui-là lit le jeton. C'est la politique de contenu qui l'en empêche.

Les en-têtes de sécurité

mesure 7 · app.py

La menace

Le navigateur est permissif par défaut : il exécute les scripts qu'on lui donne, affiche la page dans un cadre invisible, devine les types de contenu, et transmet l'URL consultée au site suivant. Chacun de ces comportements est une attaque possible.

La parade

Huit en-têtes posés sur *chaque* réponse, par un `after_request` unique — impossible d'en oublier un sur une route.

Dans le code

```
@app.after_request
def security_headers(response):
    csp = ("default-src 'self'; script-src 'self'; style-src 'self'; "
           "img-src 'self' data:; font-src 'self'; connect-src 'self';
               ↪ "
           "object-src 'none'; base-uri 'self'; form-action 'self'; "
           "frame-ancestors 'none'")
    if HTTPS:
        csp += "; upgrade-insecure-requests"
        response.headers["Strict-Transport-Security"] = \
            "max-age=31536000; includeSubDomains"
    response.headers["Content-Security-Policy"]   = csp
    response.headers["X-Content-Type-Options"]    = "nosniff"
    response.headers["X-Frame-Options"]           = "DENY"
    response.headers["Referrer-Policy"]           = "no-referrer"
    response.headers["Permissions-Policy"]        = \
        "camera=(), microphone=(), geolocation=(), payment=()"
    response.headers["Cross-Origin-Opener-Policy"]   = "same-origin"
    response.headers["Cross-Origin-Resource-Policy"] = "same-origin"
```

En-tête	Ce qu'il interdit
`Content-Security-Policy`	Charger un script, un style, une police ou une image d'ailleurs que du site lui-même. `object-src 'none'` bloque les greffons, `form-action 'self'` empêche un formulaire de poster ailleurs, `base-uri 'self'` empêche de réécrire la base des liens relatifs.
`frame-ancestors 'none'` `X-Frame-Options: DENY`	Afficher le site dans un cadre sur un autre site — la parade au *clickjacking*. Les deux en-têtes disent la même chose ; le second pour les navigateurs anciens.
`X-Content-Type-Options`	Deviner le type d'un fichier plutôt que de croire l'en-tête : un fichier téléversé ne peut pas être réinterprété en script.
`Referrer-Policy: no-referrer`	Transmettre au site suivant l'URL que l'utilisateur consultait — qui contient parfois un identifiant.
`Permissions-Policy`	Demander la caméra, le micro, la position ou le paiement. La plateforme n'en a aucun usage ; autant les refuser explicitement.
`Strict-Transport-Security`	Revenir en HTTP pendant un an, y compris sur les sous-domaines. Posé seulement si le site est réellement en TLS — le poser en HTTP local rendrait le site inaccessible.
`Cross-Origin-*`	Partager le contexte de navigation ou les ressources avec une autre origine.

Piège — `default-src 'self'` interdit les CDN — et c'est voulu

Une politique stricte a un coût concret : on ne peut plus écrire `<script src="https://cdn.tailwindcss.com">`. Tailwind est donc **précompilé** et servi depuis le site.

Le gain dépasse la sécurité : la plateforme ne dépend plus d'un service extérieur qui pourrait tomber, changer de contenu ou observer les visiteurs. Une politique de contenu qu'on affaiblit pour faire marcher un CDN ne protège plus de grand-chose — autant la tenir.

Ce que ça ne couvre pas

La CSP limite les dégâts d'une injection, elle ne l'empêche pas. La vraie défense contre le XSS reste l'échappement automatique des gabarits Jinja2, que la CSP double en dernier recours.

CHAPITRE 4

Couche 4 — Les secrets

La plateforme manipule quatre secrets : la clé de session, la clé du coffre, le mot de passe de la base, et le jeton des agents. Plus un secret par machine louée. Aucun ne doit se retrouver en clair, nulle part.

Deux clés, jamais la même

mesure 8 · `crypto.py`

La menace

Une seule clé pour tout est commode : on la perd une fois, on perd tout. Si la clé qui signe les cookies est aussi celle qui chiffre les mots de passe `root`, alors la fuite d'une clé de session — par nature plus exposée, puisqu'elle sert à chaque requête — livre l'accès à toutes les machines louées.

La parade

Deux clés distinctes, de deux natures différentes, pour deux usages qui n'ont rien à voir.

Dans le code

```
def load_secret_key() -> str:
    """Cle secrete des sessions Flask."""
    return load_or_create_key("INSACLOUD_SECRET_KEY", ".secret_key",
                              lambda: secrets.token_hex(32))

# Cle de chiffrement des mots de passe root (distincte de la cle de
    ↪ session)
VAULT = Fernet(load_or_create_key("INSACLOUD_VAULT_KEY", ".vault_key",
                                  lambda: Fernet.generate_key().decode
                                      ↪ ()))
```

Le commentaire en tête du module le dit explicitement : *« Séparer les deux garantit qu'un vol de la clé de session ne donne pas accès aux machines louées. »*

Notion — D'où viennent les clés

`load_or_create_key` suit trois pistes, dans l'ordre :

1. la **variable d'environnement** — c'est ainsi qu'Ansible les injecte depuis son Vault, en production ;
2. un **fichier local** déjà présent ;
3. à défaut, une clé **tirée au hasard**, écrite dans un fichier aussitôt

mis en `0600`.

Ce troisième cas est important : il n'existe *aucune* valeur par défaut inscrite dans le code. Une clé par défaut serait la même sur tous les déploiements, donc publique.

Ce que ça ne couvre pas

Les fichiers `.secret_key` et `.vault_key` vivent sur le disque du contrôleur. Qui lit ce disque les obtient. Un gestionnaire externe (HashiCorp Vault) les garderait hors de la machine, au prix d'un service supplémentaire à déployer et à sécuriser lui-même.

Le coffre : Fernet

mesure 9 · `crypto.py`, `services.py`

La menace

Le mot de passe `root` de chaque machine louée doit être conservé : pour l'afficher à l'utilisateur quand il le redemande, et pour le faire tourner. En clair dans la base, il tombe avec elle.

La parade

Chaque mot de passe est chiffré avant d'être écrit, par **Fernet** — la recette de la bibliothèque `cryptography` : AES-128 en mode CBC pour le chiffrement, HMAC-SHA256 pour l'authentification, et un horodatage.

Dans le code

```
def encrypt_secret(value: str) -> str:
    """Chiffre un secret avant de l'ecrire en base."""
    return VAULT.encrypt(value.encode()).decode()
```

Appelé aux trois endroits où un mot de passe naît ou change : à la création de la machine, à la régénération demandée par l'utilisateur, et à la recréation après la panne d'un nœud.

Notion — Pourquoi Fernet plutôt qu'AES « à la main »

Fernet est un format **authentifié** : il ne se contente pas de chiffrer, il signe. Un attaquant qui modifierait un octet du texte chiffré en base provoquerait un `InvalidToken` au déchiffrement, et non un mot de passe différent mais accepté. Chiffrer sans authentifier est une erreur classique, qui ouvre des attaques par altération du texte chiffré ; Fernet la rend impossible par construction.

Ce que ça ne couvre pas

Le coffre protège la base, pas la mémoire : pendant qu'il sert une page,

le processus détient le mot de passe en clair. Et la clé du coffre est sur la même machine que la base — la séparation est logique, pas physique.

La fenêtre du coffre

mesure 10 · `app.py`

La menace

Afficher en permanence les mots de passe `root` sur le tableau de bord, c'est les offrir à quiconque passe derrière l'utilisateur — ou à une session restée ouverte.

La parade

Les mots de passe sont **masqués par défaut**. Pour les voir, l'utilisateur doit ressaisir le mot de passe de *son compte* ; le coffre s'ouvre alors pour cinq minutes, puis se referme seul.

Dans le code

```
VAULT_WINDOW_MINUTES = int(os.environ.get("INSACLOUD_VAULT_WINDOW", "5"
    ↪ ))
```

Le déchiffrement n'a même pas lieu si la fenêtre est fermée : le gabarit ne reçoit pas le secret, il reçoit `None`. Un secret qui n'est pas envoyé au navigateur ne peut pas y être intercepté.

```
secret = decrypt_secret(row["root_password"]) if (vault_open and
    ↪ running) else None
```

Ce que ça ne couvre pas

Cinq minutes suffisent à une capture d'écran. La fenêtre réduit l'exposition, elle ne la supprime pas — et c'est précisément pourquoi l'authentification par clé publique (couche 5) est le meilleur chemin : elle rend le mot de passe inutile.

Ansible Vault

mesure 11 · `3_ansible/group_vars/`

La menace

Les secrets doivent arriver sur le serveur. Trois mauvaises façons de s'y prendre : les écrire en clair dans le dépôt, les taper à la main à chaque déploiement, ou les passer en variables d'environnement depuis un terminal — où ils restent dans l'historique du shell.

La parade

Un fichier chiffré en **AES-256**, versionné tel quel dans le dépôt, qu'Ansible déchiffre au moment de jouer le playbook avec une phrase de passe

fournie à l'exécution.

Dans le code

`3_ansible/group_vars/insacloud/vault.yml` contient la clé de session, la clé du coffre et le mot de passe de la base. Le rôle `webapp` vérifie même leur *forme* avant de déployer — une clé Fernet doit faire exactement 44 caractères — plutôt que de laisser l'application échouer au démarrage avec un message obscur.

La phrase de passe vit dans un fichier `.vault_pass` en `0600`, exclu du dépôt par le `.gitignore`.

> **À retenir**
>
> Le fichier chiffré est *dans* le dépôt, et c'est l'intérêt : les secrets voyagent avec le code, le déploiement reste entièrement automatisable, et il n'y a aucune étape manuelle de copie de clés — étape qui, en pratique, finit toujours par être faite de travers.

Ce que ça ne couvre pas

La phrase de passe est le dernier maillon, et elle n'est protégée par rien d'autre que les droits du fichier. Qui l'obtient obtient tout le reste.

Les secrets sur le serveur

mesure 12 · rôle `webapp`

La menace

Un secret déposé sur le serveur peut fuir par trois chemins qu'on oublie : l'unité systemd (lisible par tous), les journaux, et la liste des processus.

La parade

Les secrets sont déposés dans un **fichier séparé en `0600`**, lu par systemd via `EnvironmentFile` — jamais écrits dans l'unité elle-même, qui est en lecture publique, et jamais passés en arguments de ligne de commande, qui apparaissent dans `ps`.

Dans le code

Du côté des machines louées, le même soin : l'`entrypoint.sh` termine par

```
# Le mot de passe ne doit pas rester lisible dans l'environnement des
    ↪ processus
unset ROOT_PASSWORD SSH_PUBKEY
exec "$(command -v supervisord)" -c "$CONF"
```

Sans ce `unset`, le mot de passe `root` resterait lisible dans `/proc/1/environ` pendant toute la vie de la machine.

Ce que ça ne couvre pas

`docker inspect` montre les variables d'environnement passées au `docker run`, donc le mot de passe initial, à qui a accès au démon Docker. Mais qui a accès au démon Docker a déjà tout (chapitre 1) : ce n'est pas une fuite supplémentaire.

La recherche de secrets commis mesure 13 · `gitleaks`

La menace

Un secret commis par erreur puis retiré au commit suivant **reste dans l'historique**, consultable par quiconque clone le dépôt. Le retirer de la dernière version ne sert à rien.

La parade

`gitleaks` parcourt **tout l'historique** à chaque envoi, dans un travail dédié de la chaîne d'intégration.

Dans le code

Le travail `secrets` du fichier `.github/workflows/ci.yml`. Le `.gitignore` tient par ailleurs hors du dépôt les clés (`.secret_key`, `.vault_key`), le fichier `.env` de la pile compose, et la phrase de passe du Vault.

Ce que ça ne couvre pas

`gitleaks` reconnaît les formats connus — clés AWS, jetons GitHub, clés privées. Un secret de forme quelconque, comme un mot de passe de base de données, peut lui échapper. D'où la règle, humaine : aucun secret n'entre dans un fichier versionné, chiffré ou non.

CHAPITRE 5

Couche 5 — L'accès aux machines louées

C'est la surface la plus exposée de la plateforme : des ports ouverts, des comptes `root`, et des utilisateurs qu'on ne connaît pas.

> **Le mot de passe à usage unique** mesure 14 · `services.py`

La menace

Un mot de passe `root` par défaut, ou dérivé du nom de la machine, serait le même partout : il suffirait d'en louer une pour entrer dans toutes les autres.

La parade

Seize caractères tirés au sort par machine, affichés **une seule fois**, puis conservés chiffrés. L'utilisateur peut le régénérer à chaud sans détruire sa machine.

Dans le code

Le mot de passe est injecté au lancement (`-e ROOT_PASSWORD=...`), jamais inscrit dans l'image. Le `Dockerfile` le dit explicitement :

```
# ROOT_PASSWORD n'est VOLONTAIREMENT pas defini ici : une valeur
    ↪ inscrite dans
# l'image serait lisible par `docker inspect` et identique sur toutes
    ↪ les
# machines. La plateforme l'injecte au lancement.
```

Et si la plateforme oublie de le fournir, l'`entrypoint.sh` n'applique pas de valeur de repli connue — il en tire une au hasard :

```
if [ -z "${ROOT_PASSWORD:-}" ]; then
    ROOT_PASSWORD="$(tr -dc 'A-Za-z2-9' < /dev/urandom | head -c 16)"
fi
```

> **À retenir**
>
> Une valeur par défaut est une valeur publique. Entre « échouer » et « retomber sur une valeur connue », un système sûr échoue — ou, comme

ici, tire au hasard et le journalise.

Ce que ça ne couvre pas

Le mot de passe transite par une page web et le serveur doit le connaître pour l'afficher. C'est inhérent, et c'est la raison d'être de la mesure suivante.

L'authentification par clé publique

mesure 15 · `entrypoint.sh`

La menace

Un port SSH ouvert sur Internet reçoit des milliers de tentatives par jour. Un mot de passe de seize caractères y résiste — mais la tentative reste *possible*, et le secret doit voyager pour être utilisé.

La parade

L'utilisateur dépose sa clé publique sur son compte. Les machines qu'il loue **ensuite** la reçoivent, et leur `sshd` est reconfiguré pour refuser purement et simplement la méthode « mot de passe ».

Dans le code

```
if [ -n "$SSH_PUBKEY" ]; then
    mkdir -p /root/.ssh && chmod 700 /root/.ssh
    printf '%s\n' "$SSH_PUBKEY" > /root/.ssh/authorized_keys
    chmod 600 /root/.ssh/authorized_keys
    sed -i 's/^#\?PasswordAuthentication .*/PasswordAuthentication no/'
        ↪  \
        /etc/ssh/sshd_config
    sed -i 's/^#\?PermitRootLogin .*/PermitRootLogin prohibit-password/
        ↪ ' \
        /etc/ssh/sshd_config
fi
```

Vérifié sur une machine louée, en forçant la méthode mot de passe :

```
$ ssh -o PubkeyAuthentication=no -o PreferredAuthentications=password \
      root@127.0.0.1 -p 8096
root@127.0.0.1: Permission denied (publickey).
```

Notion — Le vrai gain : il n'y a plus de secret partagé

Avec un mot de passe, le secret existe **à deux endroits** : l'utilisateur le connaît, et le serveur doit le connaître pour le vérifier. Quiconque lit la base — ou vole la clé du coffre — obtient l'accès à toutes les machines. Avec une clé, le serveur ne détient que la **moitié publique**, qui ne sert à rien à un attaquant. Compromettre la base de données d'InsaCloud

ne donne alors *aucun* accès aux machines. Le compromis d'un système ne se propage plus à l'autre.

Ce que ça ne couvre pas

Trois limites, toutes à connaître :

— elle ne vaut que pour les machines louées **ensuite** — la clé est injectée à la création du conteneur ;

— le mot de passe `root` existe toujours sur la machine, car le terminal web et le bureau en ont besoin : la clé ferme la porte SSH, pas toutes les portes ;

— une clé privée sans phrase de passe vaut ce que vaut la session de l'utilisateur sur son poste.

Le terminal web passe par `login` mesure 16 ·

`entrypoint.sh`

La menace

Un terminal dans le navigateur, c'est un shell exposé en HTTP. Mal protégé, c'est un shell `root` offert à qui connaît l'URL.

La parade

`ttyd` ne lance pas un shell : il lance `/bin/login`, le programme de connexion du système. L'utilisateur voit l'invite `login:` puis `Password:`, exactement comme sur une console physique.

Dans le code

```
# L'authentification est faite par `login` DANS le terminal (invite "
    ↪ login:"
# puis "Password:", comme une console), et non par une boite de
    ↪ dialogue HTTP.
```

Trois bénéfices, et le dernier n'est pas évident :

1. l'authentification passe par **PAM** : temporisation entre échecs, expiration de l'invite, journalisation — gratuitement ;
2. l'expérience est familière, donc l'utilisateur n'est pas tenté de contourner ;
3. **couper l'authentification SSH par mot de passe n'enferme personne dehors** : le terminal web continue de fonctionner, puisqu'il ne passe pas par SSH. L'utilisateur qui perd sa clé garde une porte.

Piège — La version précédente utilisait Basic Auth, et c'était pire

Protégé par une authentification HTTP basique, le terminal affichait une page blanche dans plusieurs navigateurs — la négociation se faisait

avant l'ouverture de la WebSocket. Le remplacement par `login` a corrigé le défaut d'usage *et* amélioré la sécurité : Basic Auth transmet le secret à chaque requête, `login` une seule fois dans un canal déjà établi.

Ce que ça ne couvre pas

`ttyd` sert le terminal en HTTP si aucun certificat n'est monté. Sur le parc déployé, le rôle `tls_cert` fournit un certificat par nœud et `ttyd` passe en `--ssl` ; en pile locale de démonstration, le trafic est en clair sur `localhost`.

Le bureau graphique

mesure 17 · `entrypoint.sh`

La menace

Un serveur VNC exposé est une cible classique : le protocole est ancien et son authentification est faible.

La parade

Trois précautions : le port VNC n'est **jamais publié**, le mot de passe du bureau est **distinct** de celui de `root`, et le pont vers le navigateur passe en TLS dès qu'un certificat est disponible.

Dans le code

```
Xvnc :1 -geometry 1280x800 -depth 24 -rfbport 5901 -localhost \
        -SecurityTypes VncAuth -PasswordFile /root/.vnc/passwd -
            ↪ AlwaysShared
websockify $NOVNC_TLS --web $NOVNC_DIR 6080 localhost:5901
```

`-localhost` est le point essentiel : le port 5901 n'est joignable que depuis l'intérieur du conteneur. Le seul port publié est le 6080, celui de noVNC. Le mot de passe du bureau est dérivé du mot de passe `root` mais tronqué à huit caractères, et stocké dans un fichier en `0600`.

Piège — Huit caractères, et ce n'est pas un choix

Le protocole VncAuth **tronque le mot de passe à huit caractères** : c'est une limite du protocole, pas un oubli. Il a donc fallu exposer un mot de passe de bureau distinct de celui de `root`, pour ne pas laisser croire à l'utilisateur que son mot de passe de seize caractères protège aussi son bureau.

Ce que ça ne couvre pas

VncAuth est faible : un défi-réponse DES des années 1990. C'est acceptable ici pour trois raisons cumulées — la machine est éphémère, le mot de passe n'est visible qu'en ouvrant le coffre pendant cinq minutes, et le

transport est chiffré dès qu'un certificat est monté. Sur une plateforme permanente, il faudrait un proxy authentifié devant noVNC.

CHAPITRE 6

Couche 6 — Le durcissement des conteneurs

Le locataire est `root` dans sa machine : c'est le service vendu. Toute la question est de faire en sorte que ce `root`-là ne soit pas `root` sur le nœud.

Les capacités retirées

mesure 18 · `docker_ops.py`

La menace

Par défaut, Docker accorde à un conteneur une quinzaine de *capacités* du noyau Linux. Certaines sont anodines ; d'autres permettent de modifier le réseau de l'hôte, de monter des systèmes de fichiers ou de charger des modules.

La parade

On retire **tout**, puis on rend uniquement ce dont une machine Linux ordinaire a besoin pour démarrer ses services. C'est une liste blanche, pas une liste noire.

Dans le code

```
--cap-drop ALL
--cap-add CHOWN --cap-add DAC_OVERRIDE --cap-add FOWNER --cap-add
    ↪ FSETID
--cap-add SETGID --cap-add SETUID --cap-add SYS_CHROOT
--cap-add NET_BIND_SERVICE --cap-add KILL --cap-add AUDIT_WRITE
```

Capacité gardée	Pourquoi elle est nécessaire
`CHOWN`, `FOWNER`, `FSETID`	Créer les répertoires de service avec les bons propriétaires et droits (`/root/.ssh`, `/run/sshd`).
`DAC_OVERRIDE`	Permettre à `root` d'écrire là où les droits ne le prévoient pas — comportement normal de `root`.
`SETGID`, `SETUID`	`login` et `sshd` changent d'identité pour ouvrir une session : sans elles, aucune connexion.
`SYS_CHROOT`	`sshd` isole son processus non privilégié par `chroot` pendant l'authentification.
`NET_BIND_SERVICE`	Écouter sur un port inférieur à 1024 — le 22.
`KILL`	`supervisord` doit pouvoir arrêter et relancer ses services.
`AUDIT_WRITE`	PAM écrit un enregistrement d'audit à chaque connexion.

Notion — Ce qui est retiré, et qui compte

`SYS_ADMIN` (monter des systèmes de fichiers — la capacité la plus dangereuse, souvent qualifiée de « nouveau `root` »), `NET_ADMIN` (reconfigurer le réseau, poser une interface en mode espion), `SYS_MODULE` (charger un module noyau), `SYS_PTRACE` (inspecter la mémoire d'autres processus), `MKNOD` (créer un fichier de périphérique, donc accéder au disque brut). Chacune de ces capacités est une voie d'évasion documentée. Les retirer ne retire rien à l'usage légitime d'une machine louée.

Ce que ça ne couvre pas

Les capacités cloisonnent le noyau, elles ne le remplacent pas. Une vulnérabilité du noyau lui-même traverse la frontière.

Pas d'élévation de privilèges mesure 19 · `docker_ops.py`

La menace

Un binaire `setuid` à l'intérieur du conteneur peut rendre des privilèges qu'on vient justement de retirer. Beaucoup d'images en contiennent sans le savoir (`ping`, `mount`, `su`).

La parade

`--security-opt no-new-privileges:true` : le noyau refuse qu'un processus du conteneur gagne des privilèges qu'il n'avait pas au départ, quel que soit le bit `setuid` du binaire exécuté.

Dans le code

Dans la constante `CONTAINER_HARDENING`, appliquée à *toutes* les machines louées sans exception — ce n'est pas une option du formulaire.

Ce que ça ne couvre pas

Le drapeau empêche de *gagner* des privilèges. Il ne retire pas ceux que `root` possède déjà dans le conteneur.

Les limites de ressources mesure 20 · `docker_ops.py`

La menace

Un locataire lance une bombe à fourche (`:() :|:& ;:`), un mineur de cryptomonnaie ou une compilation massive. Sans limite, il prend tout le nœud et toutes les autres machines deviennent inutilisables. C'est le déni de service le plus facile à monter : il suffit d'être client.

La parade

Trois plafonds, posés au lancement.

Dans le code

```
--pids-limit 512            # nombre maximal de processus : arrete la
    ↪ bombe a fourche
--cpus 1                    # un coeur au plus
--memory 256m | 1g          # selon le mode, terminal ou bureau
--memory-reservation ...    # le minimum garanti en cas de contention
```

La distinction entre `--memory` et `--memory-reservation` est celle que le cours impose : la *limite* est un plafond dur que le conteneur ne peut pas dépasser, la *réservation* est le minimum qui lui reste garanti quand le nœud est sous pression. Sans réservation, une machine bien élevée peut être étranglée par une voisine gourmande, même si celle-ci respecte sa limite.

> **À retenir**
>
> `--pids-limit` est la plus sous-estimée des trois : une bombe à fourche ne consomme ni beaucoup de CPU ni beaucoup de mémoire par processus, mais épuise la table des processus du *noyau*, qui est partagée avec l'hôte. Les limites de CPU et de mémoire ne l'arrêtent pas.

Ce que ça ne couvre pas

Les limites valent par conteneur. Trois machines par utilisateur, c'est trois fois la limite — d'où le quota.

Un réseau dédié

mesure 21 · `config.py`, `docker_ops.py`

La menace

Sur le réseau `bridge` par défaut de Docker, tous les conteneurs se voient. Un locataire peut donc scanner et atteindre les machines des autres locataires — et les services de la plateforme elle-même.

La parade

Les machines louées sont placées sur un réseau Docker **propre au déploiement**, créé à la demande.

Dans le code

```
CONTAINER_NETWORK = os.environ.get("INSACLOUD_NETWORK", f"insacloud_{
    ↪ CLUSTER}_net")
```

Piège — `--internal` aurait coupé Internet aux locataires

La première version créait le réseau avec `--internal`, qui l'isole totalement — y compris de l'extérieur. L'intention était bonne, l'effet désastreux : plus aucune machine louée ne pouvait faire `apt-get install`, ce qui est précisément ce pour quoi on loue une machine.
Le défaut a été vu avant livraison, en essayant réellement d'installer un paquet dans une machine fraîchement louée. C'est le genre d'erreur qu'aucune relecture ne détecte, parce que le code est exactement conforme à l'intention.

Ce que ça ne couvre pas

Le réseau dédié sépare les machines louées des services de la plateforme ; il ne sépare pas les locataires *entre eux*. Un réseau par machine serait plus strict, au prix d'une complexité de gestion sans commune mesure avec le gain à cette échelle.

L'identité par étiquette

mesure 22 · `docker_ops.py`

La menace

Le Faucheur détruit les conteneurs qui ne correspondent à aucune location active. S'il les reconnaît par leur *nom*, une collision de préfixe lui fait détruire autre chose — par exemple les services de la plateforme, qui s'appellent `insacloud-web`, `insacloud-db`, `insacloud-faucheur`.

La parade

Trois étiquettes Docker, apposées à chaque machine, qui servent d'identité. Jamais le nom.

Dans le code

```
MACHINE_LABEL = "insacloud.role"       # = rented-machine
CLUSTER_LABEL = "insacloud.cluster"    # = ce deploiement
WORKER_LABEL  = "insacloud.worker"     # = le noeud qui l'heberge

MACHINE_FILTERS = [f"label={MACHINE_LABEL}={MACHINE_LABEL_VALUE}",
                   f"label={CLUSTER_LABEL}={CLUSTER}"]
```

Les deux premiers filtres sont *combinés* : une machine doit être une machine louée **et** appartenir à ce déploiement. Sans le second, le Faucheur d'une plateforme détruirait les machines d'une autre tournant sur le même démon — ce qui est arrivé en développement.

Le troisième, `insacloud.worker`, a été ajouté plus tard : il dit sur quel nœud la machine a été créée. Sur un parc de machines virtuelles, la frontière des démons suffit ; dès que deux nœuds partagent un démon, elle disparaît,

et sans ce label la réconciliation de l'un détruisait les machines de l'autre.

À retenir

Ce défaut n'a pas été trouvé en relisant le code — il a été trouvé en faisant tourner un parc à deux nœuds. Une étiquette est une *donnée portée par l'objet lui-même* ; un nom est une convention que rien ne garantit. Pour une opération destructive, la différence n'est pas théorique.

Ce que ça ne couvre pas

Un attaquant ayant accès au démon Docker peut poser lui-même ces étiquettes sur un conteneur à lui, et le faire adopter par la plateforme. Mais, une fois de plus : qui a le démon a déjà tout.

La commande Docker, sans interpréteur mesure

23 · `docker_ops.py`

La menace

L'énoncé impose de lancer les machines par la commande `docker run`. Écrite en concaténant des chaînes et passée à un shell, une telle commande est une injection en puissance : une distribution nommée `alpine; rm -rf /` suffirait.

La parade

La commande est construite en **liste d'arguments** et exécutée sans shell. Aucun interpréteur ne relit jamais les valeurs.

Dans le code

```
res = subprocess.run(cmd, capture_output=True, text=True, timeout=
    ↪ timeout)
# pas de shell=True : les arguments sont passes en liste, donc jamais
# reinterpretes par un interpreteur de commandes.
```

Et en amont, les seules valeurs d'origine utilisateur — durée, distribution, mode — sont validées : la durée est un entier borné, la distribution et le mode sont choisis dans des dictionnaires fermés. Une valeur hors de ces ensembles n'atteint jamais `docker`.

Notion — Deux barrières plutôt qu'une

La validation en amont suffirait. L'absence de shell suffirait. On fait les deux, parce que la première dépend d'un développeur qui n'oublie pas un chemin, et la seconde d'une propriété du système. Quand l'une cède, l'autre tient.

Ce que ça ne couvre pas

L'exécution sans shell ne protège pas d'un argument *valide* mais malveillant — par exemple un chemin de montage arbitraire, si un jour on laissait l'utilisateur en choisir un. La liste des arguments est entièrement construite par la plateforme : c'est ce qui doit le rester.

CHAPITRE 7

Couche 7 — Le contrôleur et les nœuds

Le contrôleur doit créer et détruire des conteneurs sur des machines distantes. C'est le lien le plus sensible de l'architecture : il porte, de bout en bout, un pouvoir équivalent à `root`.

SSH plutôt que l'API Docker

mesure 24 · `workers.py`

La menace

La façon évidente de piloter un démon Docker à distance est d'exposer son API sur un port TCP. C'est aussi la plus dangereuse : cette API n'a, par défaut, **aucune authentification**, et y accéder revient à être administrateur de la machine. Les moteurs de recherche de services exposés en recensent des dizaines de milliers, et ils sont compromis en quelques minutes.

La parade

Le transport SSH natif du client Docker. Aucun port supplémentaire n'est ouvert : SSH était déjà là, et il apporte l'authentification que l'API n'a pas.

Dans le code

```
def docker_command(worker: str = "local") -> list:
    """Prefixe de commande docker pour ce noeud."""
    host = worker_host(worker)
    if host is None:
        return ["docker"]
    return ["docker", "-H", f"ssh://{DOCKER_SSH_USER}@{host}"]
```

Le module le résume en tête : *« Rien n'est exposé sur le réseau à part SSH ; l'authentification se fait par clé. »*

> **Notion — Ce que fait réellement `docker -H ssh://`**
>
> Le client Docker ouvre une session SSH et y exécute `docker system dial-stdio` sur le nœud. L'API circule alors *dans* le tunnel SSH, authentifiée et chiffrée, sans qu'aucune socket réseau n'écoute. Si l'on avait voulu sécuriser l'API TCP, il aurait fallu monter une autorité de certification, émettre un certificat client par contrôleur et les renouveler :

beaucoup de pièces mobiles pour un résultat équivalent.

Ce que ça ne couvre pas

Qui obtient la clé SSH du contrôleur obtient le démon Docker de tous les nœuds. Cette clé est donc l'actif le plus sensible du parc — d'où la mesure suivante.

La clé de pilotage

mesure 25 · rôle `controller_key`

La menace

Une clé SSH réutilisée — celle du développeur, par exemple — étend sa portée à tout le parc, et survit au départ de son propriétaire.

La parade

Une clé **dédiée au pilotage**, générée par Ansible, déposée sur les nœuds pour un **compte de service sans mot de passe**.

Dans le code

Le rôle `controller_key` crée le compte et la paire de clés ; le rôle `worker` dépose la partie publique sur chaque nœud. Le compte `insacloud` n'a pas de mot de passe — il n'est donc joignable que par clé, et uniquement depuis le contrôleur.

Sur le parc à deux nœuds en conteneurs, le même principe est rejoué par le service `cles`, qui fabrique la paire une seule fois dans un volume partagé. L'`entrypoint` du nœud installe la partie publique et, détail qui a coûté une heure, déverrouille le compte :

```
# « adduser -D » laisse le compte VERROUILLE (un « ! » dans /etc/shadow
    ↪ ), et
# sshd refuse un compte verrouille MEME en authentification par cle. L'
    ↪ etoile
# dit « aucun mot de passe n'ouvre ce compte », ce qui est l'intention.
sed -i 's|^insacloud:!|insacloud:*|' /etc/shadow
```

Piège — Un compte sans mot de passe n'est pas un compte verrouillé

Les deux états se ressemblent et n'ont rien à voir. `!` signifie « verrouillé » : `sshd` refuse la connexion, y compris par clé. `*` signifie « aucun mot de passe ne peut ouvrir ce compte » : la connexion par clé fonctionne. Le symptôme était un `Connection closed by remote host` muet côté client ; seul le journal du serveur disait `User insacloud not allowed because account is locked`.

Ce que ça ne couvre pas

La clé n'a pas de phrase de passe : elle doit être utilisable par un service, sans intervention humaine. Sa protection repose donc entièrement sur les droits du fichier et sur la sécurité du contrôleur.

Le jeton des agents

mesure 26 · `api.py`, `worker_agent.py`

La menace

L'inscription des nœuds est dynamique : un agent appelle `POST /workers/register` et le contrôleur l'ajoute au parc. Sans contrôle, n'importe qui déclare un nœud, reçoit des machines à héberger — et donc les mots de passe de leurs locataires.

La parade

Un jeton partagé, exigé dans un en-tête à l'inscription comme au battement de cœur.

Dans le code

```
"X-Agent-Token": TOKEN        # cote agent
```

Côté contrôleur, les routes `/workers/register` et `/workers/heartbeat` refusent la requête sans jeton valide — ce qu'un test vérifie explicitement (`test_api_enregistrement_refuse_sans_jeton`). Le jeton est distribué par Ansible Vault, jamais inscrit dans le code.

L'agent, lui, refuse de démarrer sans jeton plutôt que de tenter une inscription anonyme :

```
if not TOKEN:
    log.error("INSACLOUD_AGENT_TOKEN n'est pas defini : arret.")
```

Ce que ça ne couvre pas

Un jeton partagé est le même pour tous les nœuds : il ne distingue pas les agents entre eux, et le compromettre permet d'en déclarer un faux. Une authentification mutuelle par certificat serait plus robuste ; elle demanderait une autorité de certification que le projet n'a pas.

Un nœud muet n'est jamais interprété

mesure 27 · `faucheur.py`

La menace

Le Faucheur réconcilie Docker et la base : il détruit les conteneurs inconnus et marque arrêtées les machines sans conteneur. Si un nœud ne répond pas et qu'il interprète ce silence comme « il n'y a plus rien là-bas », il déclare

mortes des machines qui tournent parfaitement — et les détruira au retour du nœud.

La parade

Un nœud injoignable est **ignoré**, pas interprété. Aucune décision n'est prise sur son compte.

Dans le code

```
if res.returncode != 0:
    log.warning("Noeud %s injoignable pour la reconciliation : %s",
                worker, res.stderr.strip()[:120])
    continue          # on ne touche a rien pour ce noeud
```

Un test dédié verrouille ce comportement : `test_noeud_injoignable_ne_detruit_rien` — *« Exigence de résilience : un worker éteint ne doit pas faire perdre l'état en base. »*

> **À retenir**
>
> C'est une décision de sécurité autant que de fiabilité. La question posée au système n'est pas « que s'est-il passé ? » mais « que puis-je affirmer ? ». Un nœud silencieux ne permet d'affirmer rien du tout, et une opération destructive exige une certitude.

Ce que ça ne couvre pas

La prudence a un coût : une machine réellement disparue reste marquée active tant que son nœud ne répond pas. C'est le bon côté sur lequel se tromper.

Le pare-feu, différencié par groupe mesure 28 · rôle `security_hardening`

La menace

Un nœud ouvre son port SSH pour le contrôleur, et des centaines de ports pour les machines louées. Le contrôleur, lui, n'a besoin que de HTTPS. Appliquer la même politique aux deux ouvre inutilement des portes.

La parade

Une politique par défaut en refus, puis des règles distinctes selon le groupe Ansible auquel la machine appartient.

Dans le code

Le rôle `security_hardening` applique, via `community.general.ufw` : politique entrante par défaut, SSH avec **limitation anti-force-brute** (`ufw limit`) et restriction d'origine, ports applicatifs, et la plage des machines

louées *uniquement sur les workers*.

```
- name: Autoriser SSH avec limitation anti-force-brute sur le port {{
    ↪ ufw_ssh_port }}
  community.general.ufw:
    rule: limit
    port: "{{ ufw_ssh_port }}"

- name: Autoriser les plages de ports des machines louees
  community.general.ufw:
    ...
```

Piège — L'ordre des rôles n'est pas indifférent

Le rôle passe **avant** le rôle `docker` : `ufw enable` recharge `iptables` et effacerait les chaînes que Docker y installe. Inversé, le pare-feu casserait le réseau des conteneurs — panne difficile à diagnostiquer, parce que les deux rôles réussissent.

Ce que ça ne couvre pas

UFW filtre le trafic entrant vers l'hôte. Docker insère ses propres règles `iptables` pour les ports publiés, qui peuvent contourner UFW selon la configuration : la plage des machines louées est donc ouverte de fait dès que Docker publie les ports.

CHAPITRE 8

Couche 8 — Les données et le réseau

Deux sujets qu'on range rarement sous « sécurité » et qui en relèvent pourtant : l'intégrité de l'état, et la confidentialité du transport.

Le cloisonnement entre locataires

mesure 29 · `database.py`

La menace

C'est la faille la plus banale des applications multi-utilisateurs : l'URL `/instances/42` affiche la machine numéro 42, et personne ne vérifie qu'elle appartient bien à celui qui la demande. On change le numéro, on voit les machines des autres — et leur mot de passe `root`.

La parade

Le filtrage par propriétaire est fait **dans la requête SQL**, pas dans le gabarit ni dans une condition d'affichage. Une donnée qui n'appartient pas à l'utilisateur ne quitte jamais la base.

Dans le code

```
requete = requete.where(Rental.user_id == user_id)
...
.where(Rental.user_id == user_id, Instance.status == INSTANCE_RUNNING)
...
.where(Archive.user_id == user_id, Archive.expires_at > utc_now())
```

La condition est présente sur les instances, sur les locations *et* sur les archives — l'archive du travail d'un autre serait tout aussi compromettante que sa machine.

À retenir

Filtrer à l'affichage laisse la donnée arriver jusqu'au gabarit : il suffit d'une page oubliée, d'une réponse JSON, d'un message d'erreur trop bavard pour qu'elle sorte. Filtrer dans la requête rend la fuite structurellement impossible. C'est le test le plus important de la suite, et il porte un nom explicite : *cloisonnement entre comptes*.

Ce que ça ne couvre pas

Le cloisonnement repose sur la discipline : chaque nouvelle requête doit porter la condition. Un modèle de données où la propriété serait imposée par le moteur (*row-level security* de PostgreSQL) l'appliquerait sans dépendre du développeur.

Les contraintes d'intégrité

mesure 30 · `models.py`

La menace

Deux machines attribuées au même port sur le même nœud : la seconde ne démarre pas, ou pire, un utilisateur se connecte à la machine d'un autre. Une vérification faite en Python ne tient pas sous la concurrence — deux requêtes simultanées peuvent lire le même « port libre » avant que l'une n'écrive.

La parade

La règle est tenue par le **moteur de base de données**, par un **index unique partiel** : le couple (nœud, port) est unique, mais seulement parmi les machines en cours d'exécution.

Dans le code

Le « partiel » est essentiel : sans lui, un port utilisé par une machine détruite resterait réservé pour toujours, et la plage finirait épuisée. La contrainte ne s'applique donc qu'aux instances `running`.

Sept tests vérifient ces règles au niveau du moteur (`tests/test_contraintes.py`) : qu'un port en conflit lève bien une `IntegrityError`, que le port est libéré quand l'instance expire, que l'identifiant et l'adresse de courriel sont uniques, que les clés étrangères interdisent une location sans utilisateur — avec `PRAGMA foreign_keys = ON` pour SQLite, qui ne les applique pas par défaut.

> **Notion — Pourquoi le moteur et pas le code**
>
> Une règle tenue par le moteur reste vraie même si une requête concurrente, un script d'administration ou un futur appel oublient de la vérifier. Une règle tenue seulement par le code applicatif ne tient que tant que personne ne contourne ce code. L'énoncé demande explicitement de « tester les contraintes d'intégrité » : la distinction est exactement celle-là.

Ce que ça ne couvre pas

Les contraintes garantissent la cohérence, pas la justesse métier. Rien n'empêche d'attribuer un port valide mais déjà pris par un service de l'hôte — d'où une vérification complémentaire côté système

(`port_is_free_on_host`).

TLS de bout en bout

mesure 31 · rôle `reverse_proxy`

La menace

Le mot de passe de connexion, le cookie de session et les mots de passe `root` circulent entre le navigateur et le serveur. En clair, ils sont lisibles par quiconque partage le réseau — un réseau d'école, typiquement.

La parade

nginx termine le TLS devant Gunicorn, avec des protocoles et des suites choisis plutôt que laissés par défaut.

Dans le code

```
listen 443 ssl http2;
ssl_protocols TLSv1.2 TLSv1.3;
ssl_prefer_server_ciphers on;
ssl_ciphers ECDHE-ECDSA-AES256-GCM-SHA384:ECDHE-ECDSA-CHACHA20-POLY1305
    ↪ :
            ECDHE-ECDSA-AES128-GCM-SHA256;
client_max_body_size 16k;
```

Trois décisions :

— **TLS 1.2 minimum** — les versions 1.0 et 1.1 sont obsolètes et vulnérables ; aucun navigateur en service ne les exige.

— **Uniquement des suites ECDHE avec AEAD** : `ECDHE` assure la *confidentialité persistante* (un enregistrement du trafic reste indéchiffrable même si la clé du serveur fuite plus tard), et GCM ou ChaCha20-Poly1305 chiffrent *et* authentifient.

— `client_max_body_size 16k` — la même borne qu'en applicatif, mais appliquée par nginx : une requête démesurée est coupée avant d'atteindre Python.

Notion — `ProxyFix`, ou pourquoi tout le monde avait la même IP

Derrière un proxy inverse, Flask voit l'adresse du proxy — `127.0.0.1` — et non celle du visiteur. Le compteur anti-force-brute par adresse compterait donc toutes les tentatives du monde sur une seule, et bloquerait tout le monde au premier attaquant. `ProxyFix` (Werkzeug) lit l'en-tête `X-Forwarded-For` posé par nginx et rétablit la vraie adresse. Une mesure de sécurité mal câblée peut se retourner en déni de service.

Ce que ça ne couvre pas

Le certificat est **auto-signé** : le navigateur avertit. C'est la seule alerte

que l'analyse dynamique signale encore, et c'est le premier élément des travaux à court terme (Let's Encrypt). Un certificat auto-signé chiffre correctement, mais ne prouve pas l'identité du serveur : il ne protège donc pas d'un attaquant capable de s'interposer.

CHAPITRE 9

Couche 9 — La chaîne DevSecOps

Toutes les mesures précédentes se dégradent : une dépendance vieillit, un développeur pressé affaiblit une règle, un secret se glisse dans un commit. La chaîne d'intégration est ce qui les maintient vraies dans le temps.

Onze travaux, à chaque envoi

Travail	Outil	Ce qu'il empêche d'entrer
`lint-python`	ruff	Du code mort, des imports inutilisés, des motifs dangereux (`shell=True`, `assert` en production). Vérifie aussi que les schémas du `README` sont à jour.
`tests-python`	pytest	Une régression fonctionnelle. 169 tests, dont le cloisonnement entre comptes, le CSRF et l'anti-force-brute.
`sast-python`	bandit	Une faiblesse *dans notre code* : secret en dur, aléa non cryptographique, appel système non maîtrisé.
`sca-dependances`	pip-audit	Une dépendance portant une vulnérabilité connue. C'est ainsi qu'a été repérée une faille de Werkzeug, corrigée en passant de 3.1.8 à 3.1.9.
`secrets`	gitleaks	Un secret commis, y compris s'il a été retiré depuis : l'outil parcourt **tout l'historique**.
`dast`	nuclei + `dast.sh`	Une faiblesse visible de l'extérieur sur l'application **en train de tourner** : entête manquant, cookie mal posé, route accessible sans session.
`lint-ansible`	ansible-lint	Un playbook non idempotent, une tâche sans nom, un module obsolète. Profil `production`, le plus strict.
`images-docker`	hadolint + Trivy	Un `Dockerfile` mal écrit, et des paquets vulnérables *dans l'image livrée* — ce que l'audit des dépendances ne voit pas.
`integration`	`integration.sh`	Une plateforme qui ne démarre pas sur une machine vierge. Les six scénarios du sujet.
`publier-images`	Syft + ghcr.io	Une image sans **inventaire de composants** (SBOM) : on doit pouvoir répondre « cette image contient-elle la bibliothèque vulnérable annoncée ce matin ? ».
`deployer`	Ansible	Un déploiement manuel, non reproductible.

Notion — SAST, SCA, DAST : trois angles, pas trois noms

SAST lit *notre* code sans l'exécuter — il voit les erreurs que nous écrivons. **SCA** confronte *les dépendances* à une base de vulnérabilités — il voit les erreurs des autres, dont nous héritons. **DAST** attaque l'application *en fonctionnement* — il voit ce qu'aucune lecture de code

ne montre : un en-tête oublié par la configuration nginx, un cookie mal posé en production.
Les trois sont nécessaires parce qu'aucun ne voit ce que les deux autres voient. La plupart des failles réelles tiennent à l'assemblage, pas à une ligne.

Le seuil de blocage

mesure 32 · `ci.yml`

La menace

Une chaîne qui signale tout sans rien bloquer devient un bruit de fond qu'on cesse de lire. Une chaîne qui bloque sur tout devient un obstacle qu'on contourne.

La parade

Un seuil explicite, justifié, différent selon l'outil.

Dans le code

`hadolint` échoue au niveau `warning`, avec trois règles écartées et chacune commentée — par exemple l'épinglage des versions de paquets, écarté parce que les dépôts Debian et Alpine ne conservent pas les anciennes versions : une image épinglée cesserait de se construire à la première mise à jour du dépôt, et c'est la version de la *distribution* qui fixe déjà l'ensemble.

Trivy ignore les vulnérabilités `--ignore-unfixed` : bloquer sur une faille sans correctif disponible n'apporte rien — on ne peut rien en faire — et noierait les failles réellement corrigeables.

À retenir

Un seuil bien choisi se reconnaît à ceci : chaque exception est écrite, datée et justifiée dans un fichier versionné. Une exception non écrite est un oubli qui se transmet.

Ce que ça ne couvre pas

Les seuils reflètent un jugement, qui vieillit. Une règle écartée aujourd'hui pour de bonnes raisons peut devenir importante ; rien ne force à les relire.

La version de Python, imposée

mesure 33 · `ci.yml`

La menace

Le projet passe les tests sur le poste du développeur et échoue au déploiement, parce que la cible n'a pas la même version du langage. C'est l'erreur la

plus banale et la plus coûteuse, et elle a frappé ce projet : une bibliothèque épinglée n'existait pas pour Python 3.10.

La parade

La chaîne force `PYTHON_VERSION: 3.10` — la version qu'Ubuntu 22.04 fournit, c'est-à-dire la cible réelle. Les tests tournent avec le Python de la production, pas celui du développeur.

Ce que ça ne couvre pas

L'alignement porte sur la version du langage, pas sur l'ensemble du système : seul le travail d'intégration, qui démarre réellement la pile, couvre le reste.

Ce que la chaîne a réellement trouvé

mesure 34 · bilan

Une chaîne d'intégration ne vaut pas par sa configuration mais par ses prises. Huit défauts ont été trouvés par la chaîne ou par l'exécution réelle, que la relecture n'avait pas vus. Les trois plus instructifs :

Un Faucheur qui détruisait sa propre plateforme. Il reconnaissait les machines louées par leur préfixe de nom ; les services de la pile s'appellent `insacloud-web`, `insacloud-db`. Correction : l'identité par étiquette (mesure 22).

Une réconciliation qui détruisait les machines des autres nœuds. Sur un parc où deux nœuds partagent un démon Docker, chacun voyait les machines de l'autre comme orphelines. Correction : l'étiquette `insacloud.worker`, et deux tests — dont un qui vérifie qu'une machine *sans* étiquette reste nettoyable, pour ne pas abandonner celles créées par une version antérieure.

Une erreur 500 une fois sur trois, au démarrage seulement. Gunicorn tourne avec `--preload` : il importe l'application *avant* de forker ses quatre workers, et l'initialisation de la base ouvre alors une connexion PostgreSQL qui se range dans le pool du processus maître. Les quatre fils en héritaient et se servaient de la **même socket** simultanément. Mesuré avant correction : trois échecs sur huit premières requêtes parallèles, à chacun des trois redémarrages d'essai ; après correction, vingt-quatre sur vingt-quatre.

Piège — Le défaut qui ne se voit qu'au premier regard

Ce dernier cas mérite d'être retenu pour sa forme : le message d'erreur (`connection in transaction status INTRANS`) n'avait *aucun* rapport avec la requête en cours, et le symptôme disparaissait tout seul après quelques appels. Un défaut qui se répare seul est un défaut qu'on croit imaginaire — et c'est exactement celui qu'aurait rencontré un visiteur ouvrant la page pour la première fois.

> Le test de non-régression a été validé en le faisant d'abord **échouer**, correction désactivée. Un test qu'on n'a pas vu échouer ne prouve rien.

CHAPITRE 10

Couche 10 — Les limites assumées

Ce chapitre est le plus important du document. Une défense dont on ignore la limite donne une fausse assurance, et une fausse assurance est pire qu'une absence de défense : on arrête de surveiller.

Les risques résiduels, et pourquoi ils sont acceptés

Risque	Pourquoi il est accepté ici, et ce qu'il faudrait en production	Gravité
Certificat auto-signé	Le trafic est chiffré, mais l'identité du serveur n'est pas prouvée : un attaquant capable de s'interposer pourrait se faire passer pour la plateforme. Seule alerte que l'analyse dynamique signale encore. Correction prévue : Let's Encrypt.	Moyenne
VncAuth	Défi-réponse DES, mot de passe tronqué à huit caractères par le protocole. Acceptable parce que la machine est éphémère, que le mot de passe n'est visible que cinq minutes et que le transport est chiffré. En production : un proxy authentifié devant noVNC.	Moyenne
`docker.sock` monté dans la pile compose	En mode mono-hôte, le conteneur de l'application reçoit la socket Docker de l'hôte — donc un pouvoir équivalent à `root` sur la machine. Le fichier `docker-compose.yml` le dit en toutes lettres : *« À n'utiliser que sur une machine de développement. »* Sur le parc déployé, ce montage **disparaît** : le contrôleur pilote les workers par SSH.	Élevée (développement seulement)
`root` dans le conteneur loué	C'est le service vendu, pas un défaut. Atténué par les capacités retirées, `no-new-privileges` et les limites de ressources. Une évasion par faille du noyau reste possible : seule une machine virtuelle créerait la vraie frontière.	Acceptée
Un seul jeton pour tous les agents	Il ne distingue pas les nœuds entre eux ; le compromettre permet d'en déclarer un faux, donc de recevoir des machines à héberger. Une authentification mutuelle par certificat demanderait une autorité de certification.	Moyenne
La reprise ne restitue pas les fichiers	Elle recrée un conteneur, elle ne déplace pas un disque. Sans stockage partagé entre nœuds, seul le service repart. C'est le premier élément des travaux à moyen terme.	Acceptée
Les clés sur le disque du contrôleur	`.secret_key` et `.vault_key` sont en `0600`, mais sur la même machine que la base. La séparation des deux clés est logique, pas physique. Un gestionnaire externe les garderait hors de la machine.	Moyenne
Pas de journalisation centralisée	Les traces d'accès aux machines louées restent sur chaque nœud. Une investigation après incident serait laborieuse, et un attaquant ayant pris un nœud peut effacer ses traces. Travaux à moyen terme.	Moyenne
Pas de second facteur	Un mot de passe compromis suffit à prendre un compte. Travaux à moyen terme.	Faible (contexte pédago-

Ce qui est hors de portée du projet

Trois choses qu'aucune mesure du document ne traite, et qu'il serait malhonnête de laisser croire couvertes :

Un attaquant déjà administrateur du contrôleur lit la base, les clés, et pilote tous les nœuds. La défense est organisationnelle — qui a accès à la machine — pas logicielle.

Une vulnérabilité du noyau Linux traverse la frontière des conteneurs, qui partagent ce noyau avec l'hôte. Le durcissement réduit la surface, il ne crée pas d'isolation matérielle.

Un déni de service volumétrique venu de l'extérieur relève de l'hébergeur. Les limites par conteneur protègent les locataires les uns des autres, pas la plateforme d'une attaque réseau.

Récapitulatif : menace, parade, preuve

Menace	Parade	Preuve
Fuite de la base	PBKDF2 salé, Fernet	Les mots de passe en base sont illisibles sans la clé du coffre.
Énumération des comptes	Empreinte factice si le compte n'existe pas	Le temps de réponse est identique dans les deux cas.
Force brute sur la connexion	5 échecs par compte, 20 par adresse, 15 min	`test_securite.py`
Vol de session	`HttpOnly`, `SameSite=Strict`, `Secure`, 2 h	En-têtes vérifiés par l'analyse dynamique.
CSRF	Jeton sur tous les formulaires, actif jusqu'en test	`test_securite.py`
Injection de script	CSP `default-src 'self'`, aucun CDN	En-tête vérifié par l'analyse dynamique.
Vol de la clé de session	Clé du coffre distincte	`crypto.py` : deux clés, deux origines.
Secret commis dans Git	gitleaks sur tout l'historique	Travail `secrets` de la chaîne.
Devinage du mot de passe d'une machine	Clé publique, `PasswordAuthentication no`	`Permission denied (publickey)`
Évasion de conteneur	`cap-drop ALL`, `no-new-privileges`	`test_docker.py` vérifie la commande construite.
Épuisement du nœud	`pids-limit`, `cpus`, `memory`	`test_docker.py`
Prise de contrôle d'un démon Docker	SSH par clé dédiée, API jamais exposée	`workers.py` : `docker -H ssh://`
Faux nœud déclaré	Jeton d'agent exigé	`test_api_enregistrement_refuse_sans_jeton`
Destruction sur un nœud muet	Le silence n'est pas interprété	`test_noeud_injoignable_ne_detruit_rien`
Accès aux machines d'un autre	Filtrage par propriétaire en SQL	`test_securite.py`, cloisonnement entre comptes
Collision de port	Index unique partiel en base	`test_contraintes.py` (7 cas)
Écoute du trafic	TLS 1.2/1.3, suites ECDHE + AEAD	Configuration nginx du rôle `reverse_proxy`.
Dépendance vulnérable	pip-audit, Trivy, SBOM	Werkzeug 3.1.8 → 3.1.9 après alerte.

À retenir

Dix-huit menaces, dix-huit parades, et pour chacune un endroit où

> aller voir. C'est ce dernier point qui compte : une mesure de sécurité qu'aucun test ne verrouille est une mesure qui disparaîtra à la prochaine modification, sans que personne s'en aperçoive.